

2. 排序與二分搜

「完全無序，沒有效率；排序完整，增刪折騰；完美和完整不是同一回事。」

(sorted array vs. balanced tree)

「上窮碧落下黃泉，姐在何處尋不見，人間測得上或下，不入地獄，就是上天；

天上地下千里遠，每次遞迴皆減半，歷經十世終不悔，除非無姐，終能相見。」

(binary search)

本章介紹排序與二分搜的基本用法與應用，同時也介紹其推廣運用，包括快速幕與折半枚舉。從這一章開始，我們會介紹到一些 C++ 的 container，這些容器都是一些別人已經寫好的資料結構，對於 APCS 這樣程度的考試來說，不使用這些容器也都可以考到五級分，但是使用這些容器往往可以簡化我們的程式，所以我們還是介紹常用的一些使用方式。如果你對程式還不是很熟練覺得學太多新的東西會負擔太大，那麼可以暫時略過你覺得比較複雜的部份。此外，這些容器的完整使用說明其實還蠻複雜的，這份教材中只會介紹簡單與常用的功能，需要更完整的說明，網路上都有技術文件可以參考，只要搜尋 C++ 就可找到。

2.1. 排序

排序是將一群資料根據某種順序由小到大排列，最常見的順序就是數字的由小到大或是由大到小，當然也可能是字元字串或是其他使用者自訂的順序。排序是非常重要的程序，計算機科學在發展初期就研發了很多種的排序演算法，這些演算法目前依然常常用在教學上當作例子與學習教材。在這裡我們主要介紹它的應用技巧而非排序演算法，對排序演算法有興趣的人可以查網路。

排序通常有幾個運用的時機：

- 需要把相同資料排在一起
- 便於快速搜尋
- 做為其他演算法的執行順序，例如 Sweeping-line, Greedy, DP。

在考試與競賽的場合，最重要的是會使用庫存的排序函數，在 C 是 `qsort()`，在 C++ 是 `sort()`，這些庫存函數都非常的有效率，如果要自己寫出這麼有效率的排序需要花費很多的時間還不一定能寫好。以下我們先介紹它的使用方式與考試競賽的使用技巧，`qsort` 的使用比較麻煩，我們建議使用 C++ 的 `sort()`。

最簡單使用 `sort()` 的方式就是針對一群數字的排序，例如陣列中的整數，傳入的第一個參數代表排序的起始位置，第二個參數是排序範圍結束的**下一個位置**。其使用方式如下面的範例程式所示範，其中我們用了一個 `random_shuffle()` 的函數，它是用來將某個範圍的順序弄亂的。請注意，C++ 中函數定義區間時，幾乎都是左閉右開區間，例如以上的 `sort()` 與 `random_shuffle()`：

```
#include <bits/stdc++.h>
using namespace std;
#define N 10
void show(int a[], int n) {
    for (int i=0;i<n-1;i++)
        printf("%3d ",a[i]);
    printf("%3d\n",a[n-1]);
    return;
}

int main() {
    int a[N], n=10;
    for (int i=0;i<n;i++) a[i]=rand()%100;
    show(a,n);
    printf("Sort entire array\n");
    sort(a, a+n);
    show(a,n);

    printf("Random shuffle\n");
    random_shuffle(a, a+n);
    show(a,n);

    printf("Sort a[3..7]\n");
    sort(a+3, a+8);
    show(a,n);
    return 0;
}
```

如果想要由大排到小，通常有兩個方法：一種是將原來的數字變號 (x 變 $-x$)，另外一種方法是自行指定比較函數。事實上 `sort()` 除了前兩個參數指定範圍之外，還可以傳入第三個參數，第三個參數是一個比較函數，這個函數負責告訴 `sort()` 排序的順序是什麼：比較函數必須傳入兩個參數，如果第一個參數要排在第二個的前面就回傳 `true`，否則回傳 `false`。這樣聽起來很複雜，看例子比較簡單，我們也一起說明多欄位資料的排序。

先看 `cmp1(int s, int t)`，它傳入兩個整數，回傳是否 $s > t$ ，意思是前面的要比較大。第 36 行 `sort(a, a+n, cmp1);` 會讓資料從大排到小。我們也可以呼叫庫存的比較函數 `greater`，如第 40 行的寫法，但這個寫法可能不容易記得。為了展示多欄位資料的排序，我們定義了一個結構 `struct point` 用來存放平面座標，並且展示了三個不同的比較函數讓資料依照不同的要求來排序。

```

00 #include <bits/stdc++.h>
01 using namespace std;
02 #define N 10
03 void show(int a[], int n) {
04     for (int i=0;i<n-1;i++)
05         printf("%3d ",a[i]);
06     printf("%3d\n",a[n-1]);
07     return;
08 }
09
10 bool cmp1(int s, int t) {
11     return s>t;
12 }
13 struct point {
14     int x,y;
15 };
16 bool cmp2(point &s, point &t) {
17     return s.x < t.x;
18 }
19 bool cmp3(point &s, point &t) {
20     return s.y > t.y;
21 }
22 bool cmp4(point &s, point &t) {
23     return s.x+s.y < t.x+t.y;
24 }
25 void showp(point a[], int n) {
26     for (int i=0;i<n-1;i++)
27         printf("(%2d, %2d) ",a[i].x, a[i].y);
28     printf("(%2d, %2d)\n",a[n-1].x, a[n-1].y);
29     return;
30 }
31 int main() {
32     int a[N], n=10;
33     for (int i=0;i<n;i++) a[i]=rand()%100;
34     show(a,n);
35     printf("from large to small\n");
36     sort(a, a+n, cmp1);
37     show(a,n);
38
39     printf("Using greater\n");
40     sort(a, a+n, greater<int>());
41     show(a,n);
42
43     point p[N];
44     for (int i=0;i<n;i++)
45         p[i].x=rand()%100, p[i].y=rand()%100;
46     printf("%d points on the plane\n");
47     showp(p,n);
48
49     printf("sorted for x from small to large\n");
50     sort(p, p+n, cmp2);
51     showp(p,n);
52     printf("sorted for y from large to small\n");
53     sort(p, p+n, cmp3);
54     showp(p,n);

```

```

55     printf("sorted for x+y from small to large\n");
56     sort(p, p+n, cmp4);
57     showp(p,n);
58     return 0;
59 }

```

多欄位資料的排序在很多地方都用得到，初學者應該要花時間了解 struct 與其排序的寫法。我們以下再說明一個偷懶的方法，考試與競賽時常碰到多欄位資料的排序，尤其是兩個欄位，例如平面 XY 座標或是線段左右端點，如果不想寫 struct，有沒有辦法偷懶呢？答案是有的。

C++ STL 中有定義好的 pair 可以用來存兩個欄位的資料，而 pair 的比較運算(<)就是兩個欄位的字典順序(lexicographic order)，也就是先比第一個欄位，如果第一欄位相同再比第二欄位。在應用時，我們只要把要排序的欄位放在第一欄位就可以了。要提醒幾件事：

- pair 的第一欄位叫做 first 而第二欄位叫做 second；
- 建構一個 pair 可很簡單的以用大括號{}，例如 p[i]={5, 3}；但如果是比較舊版的 C++ 可能並不支援這個寫法，如果不支援大括號直接建構的場合，就要用 make_pair() 的函數，在網路上看比較舊的程式碼可能會很常看到類似 make_pair(5,3) 這樣的寫法。

此外，如果超過兩個欄位呢？事實上也有做法，但這裡並不鼓勵初學者學太多庫存函數來偷懶，而且 struct 與比較函數還是應該要學的。如果有興趣的人可以自己去查詢 C++ tuple。以下看簡單的例子來了解用法。

```

00 #include <bits/stdc++.h>
01 using namespace std;
02 #define N 10
03
04 void showp(pair<int,int> a[], int n) {
05     for (int i=0;i<n-1;i++)
06         printf("(%ld, %ld) ",a[i].first, a[i].second);
07     printf("(%ld, %ld)\n",a[n-1].first, a[n-1].second);
08     return;
09 }
10 int main() {
11     pair<int,int> p[N];
12     int n=10;
13     for (int i=0;i<n;i++)
14         p[i]={rand()%10, rand()%10};
15     showp(p,n);
16
17     printf("sorted by lexicographic order\n");
18     sort(p, p+n);
19     showp(p,n);

```

```

20
21     vector<pair<int,int>> q(p, p+n); // copy p to q
22     for (auto s:q) {
23         printf("(%d,%d) ", s.first, s.second);
24     }
25     printf("\nsorted by greater order\n");
26     sort(q.begin(), q.end(), greater<pair<int,int>>());
27     for (auto s:q) {
28         printf("(%d,%d) ", s.first, s.second);
29     }
30
31     return 0;
32 }

```

第 21 行開始的最後一段我們展示了如何對 vector 使用 sort()，並且使用 greater 將它從大到小排序。vector 基本上可以看成陣列，只是長度可變，除非在處理樹狀圖與圖形問題上，通常不一定要使用，如果暫時對你太複雜，你都可以用陣列來替換它。

下面的例題是排序的一個應用，常用在某些題目前置處理，為了方便練習，我們把它寫成題目的樣子。

P-2-1. 不同的數-排序

假設有 N 個整數要被讀到一個陣列中，我們想要將這些數字排序並去除重複的數字，例如輸入的整數序列是 (5, 3, 9, 3, 15, 9, 8, 9)，這些數如從小到大排是 (3, 3, 5, 8, 9, 9, 9, 15)，去除重複者後為 (3, 5, 8, 9, 15)。寫一個函數，傳回有多少不同的數字並且將結果放在陣列中回傳。

Time limit: 1 秒

輸入格式：輸入兩行，第一行是正整數 N ， N 不超過 10 萬，第二行是 N 個整數，大小不超過 10^9 ，以空白間隔。

輸出：第一行輸出有多少相異整數，第二行由小到大輸出這些相異整數，相鄰數字之間以一個空白間隔。

範例輸入：

```

7
0 3 9 3 3 -1 0

```

範例結果：

```

4
-1 0 3 9

```

請看以下的範例程式。函數 `distinct()` 傳入兩整數陣列：`from[]` 是要傳進來的數字，`to[]` 是擺放結果的陣列，另外 `n` 是傳入數字的個數。我們不希望破壞原陣列的內容，先將資料複製到另外一個 `vector v`，並且將 `from[]` 的內容複製到 `v`。接著將 `v` 內容排序，排序後相同的數字會在一起，因此除了 `v[0]` 之外，只要是 `v[i] != v[i-1]` 的 `v[i]` 都是一個相異的數字。請留意我們以變數 `num` 儲存目前找到相異數的個數，並且 `to[num]` 就是儲存位置，將相異數字複製到 `to[num]` 之後必須將 `num` 的值增加 1，這兩個動作可以寫成一個指令來完成：

```
to[num++] = v[i];
```

```
// P_2_1 distinct number -- sort
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int distinct(int from[], int to[], int n) {
    if (n<1) return 0;
    vector<int> v(from, from+n); // copy from[] to v
    sort(v.begin(), v.end());
    to[0]=v[0];
    int num=1; // number of distinct number
    for (int i=1; i<n; i++)
        if (v[i]!=v[i-1]) // distinct
            to[num++] = v[i];
    return num;
}

int main() {
    int a[N], b[N], n, k;
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d", a+i);
    k=distinct(a,b,n);
    printf("%d\n",k);
    for (int i=0; i<k-1; i++)
        printf("%d ",b[i]);
    printf("%d\n",b[k-1]);
    return 0;
}
```

2.2. 搜尋

在一群資料中搜尋某筆資料，通常有線性搜尋與二分搜尋，線性搜尋就是一個一個找過去，在沒有順序的資料中，也只好如此。如果資料已經依照順序排好，我們可以採用二分搜尋，原因是效率好太多了。以一個 100 萬筆的資料來說，線性搜尋可能需要比對

100 萬次，而二分搜尋的 worst case 複雜度是 $O(\log(n))$ ，最多只要比較 20 次就可以找到了，也就是有 5 萬倍的效率差異，所以對很多問題是非用不可。二分搜除了自己寫之外，也有庫存函數可以呼叫，此外，C++也有一些好用的資料結構可以達到搜尋的目的，我們在這一節中會介紹以下技巧：

- 基本二分搜的寫法
- C++的二分搜函數以及相關資料結構

此外，二分搜尋不單是在一群資料中找資料，它往往也搭配其他演算法策略使用，這一部份會出現在後續的章節中。

基本二分搜的寫法

最簡單的二分搜是在一個排好序的陣列中尋找某個元素，找到了回傳 index，否則回傳找不到 (例如 -1)，這樣的二分搜好寫也不容易寫錯，以下是個範例。二分搜的重點在於 left 與 right 兩變數紀錄著搜尋範圍的左右邊界，每次取出中間位置來比較，結果是找到了或捨棄左半邊或捨棄右半邊。當 $left > right$ 表示搜尋區間已經沒有了，這時離開迴圈，因為要找的元素不存在。若初始的區間範圍是 n ，時間複雜度是 $O(\log(n))$ ，原因是區間長度每次都減半。

```
#include <bits/stdc++.h>
using namespace std;
#define N 10
// binary search x between a[left..right]
int bsearch(int a[], int left, int right, int x) {
    while (left <= right) {
        int mid = (left + right) / 2; // middle element
        if (a[mid] == x) return mid;
        if (a[mid] < x) left = mid + 1; // search right part
        else right = mid - 1; // search left part
    }
    return -1;
}

int main() {
    int p[N];
    int n = 10;
    for (int i = 0; i < n; i++)
        p[i] = rand() % 100;
    sort(p, p + n);
    for (int i = 0; i < n; i++) printf("%d ", p[i]);
    printf("\n");

    printf("search %d => return %d\n", p[7], bsearch(p, 0, n - 1, p[7]));
    int t = rand() % 100;
    printf("search %d => return %d\n", t, bsearch(p, 0, n - 1, t));

    return 0;
}
```

```
}

```

在很多時候，我們需要知道不只是某元素在不在，當它不在時，希望找到小於它的最大值的位置 (或是大於它的最小值)，這看似只要把上述程式簡單修改一下就好了，但事實上很容易寫錯。假設我們需要知道第一個大於等於 x 的位置，我們將前述程式直接修改如下。

```
// find the first >= x between a[left..right]
int bsearch(int a[], int left, int right, int x) {
    while (left <= right) {
        int mid=(left+right)/2; // middle element
        if (a[mid]==x) return mid;
        if (a[mid]<x) left = mid+1; // search right part
        else right = mid-1; // search left part
    }
    return ???; // maybe wrong
}
```

你檢查後可能發現應該要回傳 `left` 是對的，事實上很多人會弄錯，另外也會不小心寫成無窮迴圈。這個寫法並非一定不好，這裡只是希望提醒讀者這個寫法容易犯錯。比較推薦的寫法在前一章曾經提出過，不是用左右搜的方式，而是用一路往前跳的方式，你想想看，正常人如果反覆的向左向右是不是很容易昏頭，一路往前應該比較不容易搞錯。

請看以下的範例程式，傳入的是搜尋區間的大小 n ，在一開始檢查一下 `a[0]` 是否就是所要答案，如果不是，程式中就會一直保持 `a[po]<x` 這個不變性，在此特性下，只要還能往前跳就往跳，跳的距離每次把它折半。

```
// binary search the first >=x between a[0..n-1]
int jump_search(int a[], int n, int x) {
    if (a[0]>=x) return 0; // check the first
    int po=0; // the current position, always < x
    for (int jump=n/2; jump>0; jump/=2) { //jump distance
        while (po+jump<n && a[po+jump]<x)
            po += jump;
    }
    return po+1;
}
```

時間複雜度也是 $O(\log(n))$ ，因為跳的距離每次折半，而且內層的 `while` 迴圈至多執行兩次 (不會很難證明)。

C++的二分搜函數以及相關資料結構

C++中提供了許多跟搜尋有關的函數以及資料結構，在考試或比賽中，如果是適合的題目，使用庫存函數可以節省時間也避免錯誤，這裡介紹一些基本的用法。常用的二分搜相關函數有下列三個：

- `binary_search()`
- `lower_bound()`
- `upper_bound()`

其實只要學會使用 `lower_bound()`，這個函數是用來找到第一個**大於等於**某個值的位置，另外兩個都很像，`upper_bound()`找到第一個**大於**某值的位置，而 `binary_search()` 是只傳回是否找到。

與 `sort()` 一樣，函數 `lower_bound()` 允許自訂比較函數，當用在基本資料型態且以內定的比較函數時，它的用法最簡單，例如資料是整數且以小於來比較。下面的範例我們示範用於陣列與 `vector`。基本上，呼叫時以

```
lower_bound(first, last, t);
```

要求在 `[first, last)` 的範圍內以二分搜找到第一個大於或等於 `t` 的值，找到時回傳所在位置，找不到時 (`t` 比最後一個大)，回傳 `last`。以下有幾點請注意：

- 找到時回傳的是位置，要得到陣列索引值就將它減去陣列起始位置。
- 呼叫 `lower_bound()` 必須確定搜尋範圍是排好序的，如果你針對亂七八糟的資料以 `lower_bound()` 去進行二分搜，只會得到亂七八糟的結果，編譯器是不會告訴你沒排序的。
- 搜尋範圍是傳統的左閉右開區間，也就是不含 `last`；
- 找不到時回傳 `last`，通常這個位置是超過陣列的範圍，所以沒確定找到時不可以直接去引用該位置的值。

以下這個範例程式很簡單，可以把它拿來執行一下就可以了解 `lower_bound` 的基本運用。

```
// demo lower_bound for int array
// binary search the first >=x
#include <bits/stdc++.h>
using namespace std;
#define N 5

int main() {
    int p[N]={5, 1, 8, 3, 9};
    int n=5;
    sort(p, p+n);
    for (int i=0;i<n;i++) printf("%d ",p[i]);
```

```

printf("\n");
for (int i=0;i<5;i++) {
    int t=i*3;
    // search [first=p, last=p+n) to find the first >=t
    int ndx=lower_bound(p, p+n, t) - p;
    if (ndx<n)
        printf("The first >=%d is at [%d]\n",t, ndx);
    else // return the last address if not found
        printf("No one >=%d\n",t);
}
// for vector
vector<int> v(p, p+n); // copy p to vector v
for (int i=0;i<5;i++) {
    int t=i*3;
    int ndx=lower_bound(v.begin(), v.end(), t) - v.begin();
    if (ndx<n)
        printf("The first >=%d is at [%d]\n",t, ndx);
    else // return v.end() if not found
        printf("No one >=%d\n",t);
}

return 0;
}

```

與 `sort()` 一樣，當我們要使用非內定的比較函數或是資料為多欄位結構時，我們就需要自己寫比較函數，同樣地，一個偷懶的方法是使用庫存的結構 `pair`。以下的範例程式展示結構資料的二分搜以及利用 `pair` 來存放資料而逃避寫比較函數的方式

```

// demo lower_bound for struct and pair
#include <bits/stdc++.h>
using namespace std;
#define N 5
struct point {
    int x,y;
};
// compare by only x
bool pcmp(point s, point t) {
    return s.x < t.x;
}
int main() {
    point p[N];
    int n=5;
    for (int i=0;i<n;i++) p[i].x=rand()%10, p[i].y=rand()%10;
    sort(p, p+n, pcmp);
    for (int i=0;i<n;i++) printf("(%d,%d) ",p[i].x, p[i].y);
    printf("\n");
    for (int i=0;i<5;i++) {
        point t={i*3,rand()%10};
        int ndx=lower_bound(p, p+n, t, pcmp) - p;
        if (ndx<n)
            printf("Find >=(%d,%d) at [%d]=(%d,%d)\n", \
                t.x, t.y, ndx,p[ndx].x, p[ndx].y);
        else
            printf("No one >=(%d,%d)\n",t.x, t.y);
    }
    // for vector

```

```

vector<point> v(p, p+n);
for (int i=0;i<5;i++) {
    point t={i*3,rand()%10};
    auto q=lower_bound(v.begin(), v.end(), t, pcmp);
    if (q!=v.end())
        printf("Find >=(%d,%d) at [%d]=(%d,%d)\n", \
            t.x, t.y, q-v.begin(), q->x, q->y);
    else
        printf("No x>=(%d,%d)\n",t.x, t.y);
}
// using pair, default compare function
printf("Compare by x and then y\n");
vector<pair<int,int>> a;
for (point e:v) a.push_back({e.x,e.y});
for (int i=0;i<5;i++) {
    pair<int,int> t={i*3,rand()%10};
    auto q=lower_bound(a.begin(), a.end(), t);
    if (q!=a.end())
        printf("Find >=(%d,%d) at [%d]=(%d,%d)\n", \
            t.first,t.second, q-a.begin(), q->first, q->second);
    else
        printf("No x>=(%d,%d)\n",t.first, t.second);
}
printf("Compare by only x\n");
for (int i=0;i<5;i++) {
    pair<int,int> t={i*3,rand()%10};
    // set t.second to minimum to ignore comparison of y-value
    auto q=lower_bound(a.begin(), a.end(), make_pair(t.first,-1));
    if (q!=a.end())
        printf("Find >=(%d,%d) at [%d]=(%d,%d)\n", \
            t.first,t.second, q-a.begin(), q->first, q->second);
    else
        printf("No x>=(%d,%d)\n",t.first, t.second);
}

return 0;
}

```

下面一個例題是一個有趣的排序與搜尋練習，也是實際解題時經常需要用的步驟，它有個名字是「離散化」，雖然這個名字似乎不是那麼恰當，但是從以前就這麼多人這樣稱呼它。它需要用到前一個例題 P_2_1 (不同的數) 當作它的步驟。

P-2-2. 離散化 - sort

假設有 N 個整數要被讀到一個陣列中，我們想要將這些整數置換成從 0 開始依序排列的整數並且維持它們原來的大小關係，例如輸入的整數序列是 (5, 3, 9, 3, 15, 9, 8, 9)，這些數如從小到大排是 (3, 3, 5, 8, 9, 9, 9, 15)，去除重複者後為 (3, 5, 8, 9, 15)，所以我們要替換的是：

3 → 0

5 → 1

8 → 2

9 → 3

15 → 4

所以原先的序列就會變成 (1, 0, 3, 0, 4, 3, 2, 3)。

Time limit: 1 秒

輸入格式：輸入兩行，第一行是正整數 N ， N 不超過 10 萬，第二行是 N 個整數，大小不超過 10^9 ，以空白間隔。

輸出：輸出置換後的序列，兩數之間以一個空白間隔。

範例輸入：

```
7
0 3 9 3 3 -1 0
```

範例輸出：

```
1 2 3 2 2 0 1
```

如果已經了解如何處理不同的數字，加上二分搜尋就可以簡單地處理這個問題，請看以下範例。我們先呼叫剛才寫好的函數將陣列中的相異數字排序在 $b[]$ 陣列中，然後，對於原陣列 $a[]$ 中的每一個數字，將其置換成它在 $b[]$ 中的 `index` 就可以了，而要快速的在 $b[]$ 中找到 $a[i]$ ，我們用二分搜，自己寫或是呼叫 `lower_bound()` 都可以，因為 $a[i]$ 一定在 $b[]$ 中，所以我們不需要擔心找不到的情形。

```
// P_2_2 discretization -- sort
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int distinct(int from[], int to[], int n) {
    if (n < 1) return 0;
    vector<int> v(from, from+n); // copy from[] to v
    sort(v.begin(), v.end());
    to[0]=v[0];
    int num=1; // number of distinct number
    for (int i=1; i<n; i++)
        if (v[i]!=v[i-1]) // distinct
            to[num++] = v[i];
    return num;
}

int main() {
    int a[N], b[N], n, k;
    // input data
    scanf("%d", &n);
    for (int i=0; i<n; i++)
```

```

        scanf("%d", a+i);
        // sort distinct number to b
        k=distinct(a,b,n);
        // replace number with its rank
        for (int i=0;i<n;i++) {
            a[i] = lower_bound(b, b+k,a[i]) - b; // always found
        }
        // output
        for (int i=0;i<n-1;i++)
            printf("%d ",a[i]);
        printf("%d\n",a[n-1]);
        return 0;
    }

```

檔案中範例程式 P_2_2b.cpp 則是以自己寫二分搜的方式來做，這裡就不把它列出來了。

C++可供搜尋的容器(set/map) (*)

C++的 set 可以看成一個容器，它其實是一個稱為平衡的二元搜尋樹(BST, Binary Search Tree)的動態的資料結構，所謂動態資料結構指的是可以(有效率的)做插入與刪除。在這裡我們著重在如何使用它而非它的原理，我們只要知道在 set 中插入、刪除與搜尋一個元素都可以在 $O(\log(n))$ 的時間完成，此外它是有順序的，如果我們從頭到尾做一個歷遍(Traversal)，它經過的資料的順序就是由小到大排列的，它是一個非常有效能而且容易使用的技術，值得花一些時間加以練習。

在以下的範例中，我們展示了 set 的基本用法，包含如何宣告、歷遍、插入資料、刪除資料、以及搜尋資料。有件事情要特別注意，set 和一般中學數學中的一般集合一樣，不允許多重元素，如果 set 中插入已經有的元素，事實上該插入的動作是不成功的，set 的 insert() 有多種形式，其中有的會回傳是否成功，需要了解的人可以參考網路上的技術文件。

```

// demo set
#include <bits/stdc++.h>
using namespace std;
#define N 10
#define P 10

int main() {
    set<int> S; // a set S for storing int
    printf("Insert: ");
    for (int i=0;i<N;i++) {
        int t=rand()%P;
        S.insert(t); // insert an element into S
        printf("%d ",t);
    }
    printf("\nTraversal after insertion: ");
    for (auto it=S.begin(); it!=S.end(); it++) {

```

```

        printf("%d ", *it); // iterator as a pointer
    }
    printf("\nAnother traversal: for (int e:S)");
    for (int e: S) { // for each element e in S, do ...
        printf("%d ", e);
    }
    printf("\nClear and re-insert data:\n");
    S.clear(); // clear set t empty
    for (int i=0;i<N;i++)
        S.insert(i*5);
    for (int e: S)
        printf("%d ", e);
    printf("\n");
    // to find if an element in the set
    auto it=S.find(15);
    if (it!=S.end()) // return end() when not found
        printf("find 15 in S\n");
    else printf("15 is not in S\n");
    int x=15;
    printf("After S.erase(15)\n");
    S.erase(x); // erase element of value x
    it=S.find(x);
    if (it!=S.end())
        printf("find %d in S\n",x);
    else printf("%d is not in S\n",x);
    // find lower_bound, the first one >=x
    it=S.lower_bound(x);
    if (it!=S.end())
        printf("lower_bound of %d is %d\n", x, *it);
    else printf("no lower_bound of %d\n",x);
    // find upper_bound, the first one >x
    x=(N-1)*5;
    it=S.upper_bound(x);
    if (it!=S.end())
        printf("upper_bound of %d is %d\n", x, *it);
    else printf("no upper_bound of %d\n",x);

    return 0;
}

```

除了 `set` 之外，C++ 中有幾個與它很類似資料結構，包括 `multiset` 以及 `map` 與 `multimap`。顧名思義，`multiset` 與 `set` 的差別在於允許多重元素，也就是相同的元素可以有多個。而 `map` 與 `set` 其實是相同的資料結構，只是它比 `set` 多提供了一個欄位，也就是說它的每一個元素是一個兩個欄位的 `pair` 資料類型，第一個欄位 (名字是 `first`) 是 `key`，第二個欄位 (`second`) 可以用來存放其它想要的資料，所以它的用途更廣。`map` 與 `set` 一樣不可以有相同 `key` 的元素，如果需要重元素則可以使用 `multimap`。`map` 還有一個特殊的地方是它可以使用類似陣列的 `[]` 表示式，在下面的範例中我們也示範了這樣的用法。

```

// demo multi-set, map
#include <bits/stdc++.h>
using namespace std;
#define N 10

```

```

#define P 10

int main() {
    multiset<int> S;
    printf("Insert: ");
    for (int i=0;i<N;i++) {
        int t=rand()%P;
        S.insert(t);
        printf("%d ",t);
    }
    printf("\nTraversal after insertion: ");
    for (int e: S) {
        printf("%d ", e);
    }
    printf("\nClear and re-insert data:");
    S.clear(); // clear set
    vector<int> v({5,5,2,3,7,7,8});
    S.insert(v.begin(), v.end());
    for (int e: S)
        printf("%d ", e);
    printf("\nAfter S.erase(5): ");
    S.erase(5);
    for (int e: S)
        printf("%d ", e);
    printf("\nAfter erase one of 7: ");
    auto it=S.find(7);
    if (it!=S.end())
        S.erase(it);
    for (int e: S)
        printf("%d ", e);
    printf("\n");
    // demo map
    map<char, int> M;
    char str[100]="a demo of c++ map", ch;
    int len=strlen(str);
    for (int i=0; i<len; i++) M[str[i]]+=1;
    printf("\nAfter insert %s: ",str);
    for (auto e: M)
        printf("(%c:%d)", e.first, e.second);
    printf("\n");
    ch='x';
    auto mit=M.find(ch);
    if (mit==M.end())
        printf("No %c in M\n",ch);
    else printf("count(%c)=%d\n",ch,mit->second);
    ch='m';
    printf("M[%c]=%d\n",ch,M[ch]);
    ch='y';
    printf("M[%c]=%d\n",ch,M[ch]);
    M['y'] = 5;
    printf("After M['y']=5, M[%c]=%d\n",ch,M[ch]);
    printf("After erase('y'), ");
    M.erase(ch);
    mit=M.find(ch);
    if (mit==M.end())
        printf("No %c in M\n",ch);
    M[ch];
    printf("After M['y'];, M[%c]=%d\n",ch,M[ch]);
}

```

```

    return 0;
}

```

上面的範例中我們特別示範了 `multiset` 中刪除元素的兩種不同方法，刪除相同鍵值的所有元素或是刪除某一個元素。C++ 是物件導向的程式語言，往往一個相同的函數名字可以有多種不同的呼叫方式，這與傳統 C 語言有很大的差異，好處是使用起來更方便，但壞處是用錯了會不知道，因為 `compiler` 往往抓不到這一類錯誤，它算是語意的錯誤而非語法錯誤。以下再舉一個容易被誤用的例子，請看下面的程式。

這支程式其實沒有要做甚麼事情，只是建構一個很大的 `set`，然後在這個 `set` 中做 100 次的 `lower_bound()` 搜尋。`set` 的搜尋應該是程式中的第一個寫法，在程式中我們寫了第二個寫法，那其實是用在陣列或 `vector` 上的 `lower_bound` 的寫法，兩邊名字一樣讓我們好記，但是它其實是不同的，壞就壞在第二種寫法對 `compiler` 來說並不算錯誤，只是，它跑起來很慢。你可以試試看，執行時間相差非常的多。

```

// misusing lower_bound
#include <bits/stdc++.h>
using namespace std;
#define N 500000
#define P 1000000009
int ran30(int n) {
    if RAND_MAX < 40000
        return ((rand() << 15) | rand()) % n;
    else
        return rand() % n;
}
int main() {
    set<int> S;
    for (int i=0; i<N; i++)
        S.insert(ran30(P));
    vector<int> v;
    int n=100;
    for (int i=0; i<n; i++) v.push_back(ran30(P));
    clock_t t1, t2;
    t1=clock();
    int total=0;
    for (int x:v) {
        auto f=S.lower_bound(x);
        if (f!=S.end())
            total=(total+*f)%P;
    }
    t2=clock();
    printf("total=%d, time=%f\n", total, (float)(t2-t1)/CLOCKS_PER_SEC);
    // misusing lower_bound
    t1=clock();
    total=0;
    for (int x:v) {
        auto f=lower_bound(S.begin(), S.end(), x);
        if (f!=S.end())
            total=(total+*f)%P;
    }
}

```



```

t2=clock();
printf("bad using lower_bound, total=%d, time=%f\n", \
      total, (float) (t2-t1)/CLOCKS_PER_SEC);
return 0;
}

```

上面這支程式中還有一點可以提醒：為什麼寫了一個奇怪的副程式 `ran30()` 來產生亂數，而不直接用 `rand()` 就好呢？其實是因為目前某些電腦上還有很多 C++ 編譯器的版本的 `rand()` 函數是 15-bit 的隨機亂數。這樣寫可以讓在不同編譯器下都可以達到足夠大的亂數範圍。

接下來我們以前面看過的例題 P_2_2 來做 set/map 的練習，就是做離散化。前面用的是排序以及二分搜，事實上用 set/map 也是很好寫。

P-2-2C. 離散化 - set/map (*)

題目敘述請見 P-2-2。

我們利用 set/map 不可以重複 key 而且也是從小排到大的特性，只要將所有的資料裝進去，就自然可以達到第一個步驟找出相異數字的目的，在第二步驟的替換工作方面，因為不像陣列可以直接以 index 來替換，我們在開始查詢前，先從小到大走一遍，設定每一個值要替換的值，為了這個目的，我們需要用 map 來做，因為要把替換的值（相異數的排序位置）放在 map 的第二欄位。以下是範例程式。

```

// P_2_2 discretization -- map
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int main() {
    int a[N], n, k;
    // input data
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d", &a[i]);
    map<int, int> S;
    for (int i=0; i<n; i++)
        S[a[i]] = 0; // insert a[i] and set rank=0
    int r=0;
    // traversal and set rank in second
    for (auto it=S.begin(); it!=S.end(); ++it) {
        it->second = r++;
    }
    // replace number with its rank
    for (int i=0; i<n; i++) {
        a[i] = S.find(a[i]) -> second; // always found
        // find() return the iterator, then take the rank
    }
}

```

```

        // or S.lower_bound(a[i]) -> second;
    }
    // output
    for (int i=0;i<n-1;i++)
        printf("%d ",a[i]);
    printf("%d\n",a[n-1]);
    return 0;
}

```

2.3. 其他相關技巧介紹

在這一節中，我們將介紹以下主題：

- Bitonic sequence 的搜尋
- 快速冪

Bitonic sequence 的搜尋

Bitonic sequence 是指先遞增後遞減 (或者先遞減再遞增) 的序列，也有人叫它一山峰 (一山谷) 序列。在單調遞增序列 (函數) 中可以用二分搜來找第一個超越 0 值或某值的位置，那麼對於 Bitonic 序列是否可以類似二分搜有效率地找到極值呢？例如在一山谷的序列中找最小值。答案是有的，常用的方法有兩種：三分搜以及差分二分搜。

與二分搜一樣，我們始終維護一個搜尋區間，二分搜是每次將區間二等分，一個比較之後刪除一半。三分搜是將區間三等分，如果 $1/3$ 的位置大於 $2/3$ 的位置，可以丟掉左邊的 $1/3$ ；反之，如果 $2/3$ 的位置較大，則可以丟棄右邊 $1/3$ 的區間。如此每次可以將區間長度減少 $1/3$ ，在 $O(\log(n))$ 次比較可以找到最低點。

另外一個方法更簡單，若序列 f 為先遞減再遞增 (一山谷)，則差分

$$g[i] = f[i] - f[i-1]$$

在前半段是負的，在後半段是正的，只要以二分搜找到差分序列第一個大於等於 0 的位置就是 f 的最小值。

快速冪

所謂快速冪是如何快速計算 x^y 的方法，這裡要探討的不是浮點數的運算，通常是針對整數，而這個數字通常都大到超過整數變數的範圍，所以通常都是求模 (mod) P 的運算，也就是給定 x , y , P ，要計算 $x^y \pmod{P}$ 。庫存函數中計算指數的函數都是浮點數的運算，會有運算誤差，所以不能用。

P-2-3. 快速冪

輸入正整數 x, y , 與 p , 計算 $x^y \pmod{p}$ 。 x, y, p 皆不超過 $1e9+9$ 。例如 $x=2, y=5, p=11$, 則答案是 10。

Time limit: 1 秒

輸入格式：輸入 x, y , 與 p 在同一行, 以空白間隔。

輸出格式：輸出計算結果。

範例輸入：

2 5 11

範例輸出：

10

計算指數最直接的方法就是按照定義一個一個的乘, 如下列的程式碼, 提醒一下, 每次運算後都要取餘數, 否則有 overflow 的可能。

```
t = 1;
for (int i = 0; i < y; i++)
    t = (t * x) % p;
```

但是這個方法效率太差, 如果 y 是 10 億就要執行 10 億次乘法運算。前面介紹二分搜逐步縮減跳躍距離的方法, 以類似的概念, 我們可以設計出只要 $\log(y)$ 次乘法的方法, 這個方法以遞迴的形式最簡單。在以下範例中我們同時展示遞迴與迴圈兩種寫法。

```
// find x^y mod P, 32-bit positive int x, y, p
#include <stdio>
typedef long long LL;
LL exp(LL x, LL y, LL p) {
    if (y==0) return 1;
    if (y & 1) return (exp(x, y-1, p) * x) % p;
    // otherwise y is even
    LL t = exp(x, y/2, p);
    return (t * t) % p;
}

LL exp2(LL x, LL y, LL p) {
    LL t = 1, xi = x, i = 1; // t is result, xi = x^(2^i)
    while (y > 0) {
        if (y & 1) // odd, (i-1)-bit of y = 1
            t = (t * xi) % p;
```

```

        y>>=1;
        xi=(xi*xi)%p;
        i=i*2; // i is useless, for explanation
    }
    return t;
}

int main() {
    long long x, y, p, res;
    scanf("%lld%lld%lld", &x, &y, &p);
    printf("%lld\n", res=exp(x, y, p));
    if (res!=exp2(x, y, p))
        fprintf(stderr, "different result");
    return 0;
}

```

遞迴的寫法很好懂，若 y 是奇數則先遞迴求出 $y-1$ 次方後再乘一次 x ；若 y 是偶數則求出 $y/2$ 次方後自乘，終止條件為 $y=0$ 時結果是 1。遞迴的版本中， xi 是每次自乘的結果，所以它的內容是 x 的 1 次方、2 次方、4 次方、8 次方、... 這樣的方式變化，將 y 以二進制來看，對每一個 bit 為 1 的位置，把對應項的 x 次方乘到結果中就是答案。舉例來說，若 $y=19$ ，二進制是 10011，就是要計算

$$x^{19} = x^{16} * x^2 * x^1。$$

兩種寫法的效率差不多，都是很快方法，遞迴版本尤其容易記。以下是一個稍加變化的習題，給一個提示：當 x 大到超過 long long 的時候，我們需要先求 x 除以 p 的餘數，而除以 p 的餘數可以一位一位的累加計算。例如 $x=123$ ， $p=5$ ， x 可以看成 $(1*10+2)*10+3$ ，所以 x 除以 p 的餘數也就等於 $((((1*10\%p+2)\%p)*10)\%p+3)\%p$ 。

Q-2-4. 快速幂--200 位整數

題目與輸入皆同於 P-2-3，但 x 的範圍是不超過 200 位的正整數

範例輸入：

123456789012345678901234567890 5 11

範例輸出：

10

快速計算費式數列

利用快速幂我們可以計算費式數列的第 n 項。

Q-2-5. 快速計算費式數列第 n 項

令 $f[0]=0$, $f[1]=1$, 以及 $f[n]=f[n-1]+f[n-2]$ for $n>1$ 。輸入非負整數 n , 請輸出 $f[n]$ 除以 p 的餘數, $p=1000000007$ 。 $n<2^{31}$ 。

Time limit: 1 秒

輸入格式：輸入可能有多行，每一行有一個整數是一筆測資，最後一行以 -1 代表結束，不需要處理該筆測資。

輸出格式：每一行依序輸出計算結果。

範例輸入：

```
6
123456789
100
-1
```

範例輸出：

```
8
62791945
687995182
```

Q_2_5 說明：我們可以將費式序列的定義寫成以下矩陣的形式

$$\bullet \begin{bmatrix} f[n] \\ f[n-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \times \begin{bmatrix} f[n-1] \\ f[n-2] \end{bmatrix}, \begin{bmatrix} f[1] \\ f[0] \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

將上式迭代展開，因為矩陣滿足結合律，所以可以得到

$$\bullet \begin{bmatrix} f[n] \\ f[n-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \times \begin{bmatrix} f[1] \\ f[0] \end{bmatrix}$$

如果我們可以很快的算出那個矩陣 A 的 $n-1$ 次方，就可以得出

$$f[n] = A^{n-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix}。$$

2.4. 其他例題與習題

排序與搜尋大多都會與其他的演算法結合，成為某個解題方法中的一個步驟，單獨考排序的並不多。以下舉一些例子，先看一個很常見的基本問題。

P-2-6. Two-Number problem

假設 A 為 m 個相異整數的集合， B 為 n 個相異整數的集合，而 K 是一個整數。請計算有多少對 (a, b) 的組合滿足 $a \in A, b \in B$ 且 $a+b = K$ 。

Time limit: 1 秒

輸入格式：輸入可能有多行，第一行有三個整數 m, n 與 K ，第二行有 m 個整數是 A 中的元素，第三行有 n 個整數 B 中的元素一筆測資。同一行相鄰數字間以空白間隔。兩集合元素個數均不超過 10 萬，整數的絕對值不超過 10 億。

輸出格式：輸出組合個數。

範例輸入：

```
3 4 2
1 6 -3
5 1 -1 -3
```

範例輸出：

```
2
```

這一題有多個解法，基本的想法是排序後搜尋，有下列幾種作法：

- 將 A 排序後，對 B 中的每一個 b ，以二分搜在 A 中尋找 $K-b$ 。
- 將 A 與 B 分別排序後，對 A 中的每一個 a ，在 B 中以滑動的方式找 $K-a$ 。
- 把 A 放進 `set` 來做
- 把 A 放進 `unordered_set` 來做

由於作法類似，我們省略過第一與第四種。以下是第二種的範例程式，在 a 與 b 分別排序後，由前往後對於每一個 $a[i]$ ，以 `while` 迴圈由後往前找到 b 中第一個小於等於 $k - a[i]$ 的位置，請注意，由於 $a[i]$ 是由小到大，因此 j 每次不必從最尾端重新開始，只要從前一次停下的位置開始就可以了，所以如果不計排序，這個程式的複雜度是 $O(m+n)$ 。這個在陣列中維護兩個位置的方法，也有人稱為 `Two-pointers method` (雙指針方法)，它雖然用了雙迴圈但複雜度並不是平方，類似的情形我們在下一章會看到更多例子。

```
// p_2_6a find a+b=k
#include <bits/stdc++.h>
using namespace std;
```

```

#define N 100010
int a[N], b[N];

int main() {
    int m, n, k, i;
    scanf("%d %d %d", &m, &n, &k);
    for (i=0; i<m; i++) scanf("%d", &a[i]);
    for (i=0; i<n; i++) scanf("%d", &b[i]);
    sort(a, a+m); // sort a from small to large
    sort(b, b+n); // sort b from small to large
    int j=n-1; // index of b, from n-1 to 0
    int ans=0;
    for (i=0; i<m; i++) { // each a[i]
        while (j>0 && b[j]>k-a[i]) // backward linear search
            j--;
        if (a[i]+b[j]==k) ans++;
    }
    printf("%d\n", ans);
    return 0;
}

```

接下來看利用 set 的做法，請看以下範例程式，我們可以注意到，在這個例子上我們甚至不需要用到任何陣列。

```

// p_2_6b find a+b=k, using set
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int main() {
    int m, n, k, i, t;
    scanf("%d %d %d", &m, &n, &k);
    set<int> S;
    // read A into set
    for (i=0; i<m; i++) {
        scanf("%d", &t);
        S.insert(t);
    }
    int ans=0;
    for (i=0; i<n; i++) { // for each t in B
        scanf("%d", &t);
        if (S.find(k-t)!=S.end()) // search k-t in A
            ans++;
    }
    printf("%d\n", ans);
    return 0;
}

```

接下來是一個與前者類似但較複雜一點的習題。

Q-2-7. 互補團隊 (APCS201906)

前 m 個英文大寫字母每個代表一個人物，以一個字串表示一個團隊，字串由前 m 個英文大寫字母組成，不計順序也不管是否重複出現，有出現的字母表示該人物出現在團隊中。兩個團隊沒有相同的成員而且聯集起來是所有 m 個人物，則這兩個團隊稱為「互補團隊」。輸入 m 以及 n 個團隊，請計算有幾對是互補團隊。我們假設沒有兩個相同的團隊。

Time limit: 1 秒

輸入格式：第一行是兩個整數 m 與 n ， $2 \leq m \leq 26$ ， $1 \leq n \leq 50000$ 。第二行開始有 n 行，每行一個字串代表一個團隊，每個字串的长度不超過 100。

輸出格式：輸出有多少對互補團隊。

範例輸入：

```
10 5
AJBA
HCEFGGC
BIJDAIJ
EFCDHGI
HCEFGA
```

範例輸出：

```
2
```

解題提示：與前面的例題在結構上是相似的，前面是找數字，這裡是找集合。因為字串中有重複的字母而且沒有照順序，我們需要將每一個集合表示成唯一的表式方式才能夠有效的搜尋。有兩個方法可以做：

- 將每個字串去除重複字母且裡面的字母是由小到大排列的。例如 AJBA 就改成 ABJ，HCEFGGC 就改成 CEF GH。
- 以一個整數表示一個集合，第 i 個 bit 設為 1 代表第 i 個字母在集合中，否則為 0。此法在第一章窮舉子集合時也介紹過。

第二種方法要比第一種方法更快，因為數字的搜尋要比字串來得快。以下的程式片段可以將一個字串轉換成對應以及互補集合的整數：

```
//transforming a string to corresponding int
ff = (1<<m) - 1; // bits 0~(m-1) are 1
scanf("%s",s);
int len=strlen(s), team=0;
```



```
for (int j=0;j<len;j++) // 1 for existing
    team |= 1<<(s[j] - 'A'); // set bit to 1
complement = ff - team; // int of the complement
```

以下的習題我們用來展示如何求 Modular multiplicative inverse，就是在模運算下的乘法反元素，有些人稱為「模反元素」或「模逆元」。

對於任何一個正整數 a ，它的模 P 乘法反元素就是滿足 $(a * b) \% P = 1$ 的整數 b ，這裡的 $\%$ 就是取餘數運算。計算 a 的模逆元是一個很重要的運算也有許多運用，最簡單的方法是如以下的窮舉測試：

```
for (b=1; b<P; b++)
    if ((a * b) % P == 1) {
        // b is an inverse
    }
```

窮舉法的問題在於效率太差。在許多運用場合 P 是一個質數，而且探討的整數範圍都只在 $0 \sim P-1$ 。在這些假設下有一個重要的數學性質可以幫助我們快速的計算模逆元（費馬小定理）：「若 P 為質數，對任意正整數 a ， $(a^{P-2} \% P)$ 是 a 在 $[1, P-1]$ 區間的唯一乘法反元素。」根據此性質搭配快速幂就可以用 $O(\log(P))$ 個運算計算出模逆元。另一種求模逆元的方法是用 Extended Euclidean algorithm，這裡就不介紹了。

Q-2-8. 模逆元 (*)

輸入 n 個正整數，以及一個質數 P ，請計算每一個輸入數的模逆元。輸入的正整數的大小不超過 P ， $P \leq 1000000009$ ， $0 < n < 10$ 。

Time limit: 1 秒

輸入格式：第一行是 n 與 P ，第二行 n 個整數，同行數字以空白間隔。

輸出格式：依照輸入順序輸出每一個數的模逆元，相鄰數字間間隔一個空白。

範例輸入：

```
3 7
3 4 1
```

範例輸出：

```
5 2 1
```

註：費馬小定理屬於數論中的定理，應該不在 APCS 考試的背景知識內，但如果題目中給予提示說明，則模逆元並非屬於一定不可以考的範圍。

透過下面的例題，我們要來介紹折半枚舉的技巧，這一題算是有難度的題目。

P-2-9. 子集合乘積 (折半枚舉) (@@)

輸入 n 個正整數 $A[1..n]$ ，以及一個質數 P ，請計算 A 中元素各種組合中，有多少種組合其相乘積除以 P 的餘數等於 1。每個元素可以選取或不選取但不可重複選， A 中的數字可能重複。 $P \leq 1000000009$ ， $0 < n < 37$ ，且假設 A 中元素皆小於 P 。

Time limit: 1 秒

輸入格式：第一行是 n 與 P ，第二行 n 個整數是 $A[i]$ ，同行數字以空白間隔。

輸出格式：滿足條件的組合數，因為數字可能太大，請輸出該組合數除以 P 的餘數。

範例輸入：

```
5 11
1 1 2 6 10
```

範例輸出：

```
7
```

說明乘積等於 1 的組合有：(1)，(1)，(1,1)，(2,6)，(1,2,6)，(1,2,6)，(1,1,2,6) 共 7 種

我們可以透過枚舉所有的子集合來測試哪些的組合乘積為 1。在第一章我們說明過以遞迴來枚舉所有子集的方法，他的效率是 $O(2^n)$ ，在這裡 n 可能達到 36，顯然無法在一秒內完成。以下的範例程式是一個修改的方法，在相同的乘積多的場合他的效率會有所改善，但不足以通過這一題所有測資。這個程式的方法也很直接，我們逐一考慮每一個元素，以一個 map $M1$ 來記錄著目前所有可能的子集合乘積，相同的乘積只記錄一筆，但記錄能產生此乘積的子集合個數。對於下一個元素 $a[i]$ ，把原有的所有可能在乘上此新元素，就是新的可能乘積。在計算過程，新的乘積不能直接加到原來的裡面，否則無法分辨新舊。因此要利用一個 map $M2$ 來暫存，當一個元素處理完畢後交換 $M1$ 與 $M2$ ，以便下一個元素時再從 $M1$ 計算到 $M2$ 。

```
// subset product = 1 mod P, slow method
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
```

```

int main() {
    int i, n;
    LL p, a[50];
    scanf("%d%lld", &n, &p);
    for (i=0;i<n;i++)
        scanf("%lld", &a[i]);
    map<LL,LL> M1; // (product, number)
    M1[a[0]]=1; // The first element appear once
    for (i=1;i<n;i++) { // for each element
        // compute from M1 to M2
        map<LL,LL> M2(M1); // copy M1 to M2
        for (auto e:M1) {
            LL t = (e.first*a[i])%p;
            M2[t] = (M2[t] + e.second)%p;
        }
        M2[a[i]] = (M2[a[i]] + 1)%p; // for {a[i]}
        M1.swap(M2);
    }
    printf("%lld\n", M1[1]);
    return 0;
}

```

在這種場合我們可以用折半枚舉來做的更有效率一點。

對於 n 個數字，我們先將他任意均分為兩半 A 與 B ，我們要找的解 (子集合乘積等於 1) 有三種可能：在 A 中、在 B 中、以及跨 AB 兩端 (包含兩邊的元素)。我們對 A 與 B 分別去窮舉它們的子集合乘積，可以找到在 A 中與在 B 中的解。對於跨兩邊的解，我們以下列方式計算：將其中一邊 (例如 B) 的所有子集合乘積予以排序後，我們對每一個 A 的子集合乘積 x ，在 B 的子集合乘積中去搜尋 x 的模逆元 (使得 $xy = 1$ 的 y)。

有一點必須留意，我們要把 B 的子集合乘積中，將相同的乘積予以合併，原因是這一題我們需要找出組合數，若相同元素太多，一個 x 需要搜尋很多的模逆元就會太花時間。至於 A 那一邊，合併也可以提升效率，但 worst case 複雜度沒有影響。我們可以算一下整個時間複雜度：對 $n/2$ 個元素窮舉子集合需要 $O(2^{n/2})$ ，兩邊都做所以乘以 2，排序需要 $O(n \cdot 2^{n/2})$ ，接著對 $2^{n/2}$ 個數字在 $2^{n/2}$ 個數字中做二分搜，需要的時間是 $O(n \cdot 2^{n/2})$ ，所以總時間複雜度是 $O(n \cdot 2^{n/2})$ ，這比起單純窮舉 $O(2^n)$ 要快得多了。以下是範例程式，其中求模逆元是前面介紹過的快速冪方法。

```

// subset product = 1 mod P, O(n*2^(n/2)), sort
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
LL sa[1<<19], sb[1<<19]; // subset product of a and b

// generate all products of subsets of v[]
// save result in prod[], return length of prod[]
int subset(LL v[], int len, LL prod[], LL p) {
    int k=0; // size of prod[]

```

```

    for (int i=0;i<len;i++) {
        for (int j=0;j<k;j++) { // (each subset)*v[i]
            prod[k+j]=(prod[j]*v[i])%p;
        }
        prod[k+k]=v[i]; // for subset {v[i]}
        k += k+1;
    }
    return k;
}

// find x^y mod P
LL exp(LL x, LL y, LL p) {
    if (y==0) return 1;
    if (y & 1) return (exp(x, y-1,p)*x)%p;
    // otherwise y is even
    LL t=exp(x, y/2, p);
    return (t*t)%p;
}

int main() {
    int i, n;
    LL a[30], b[30]; // input data
    LL p;
    scanf("%d%lld", &n, &p);
    int len_a=n/2;
    int len_b=n-len_a;
    for (i=0;i<len_a;i++) // half in a
        scanf("%lld", &a[i]);
    for (i=0;i<len_b;i++) // half in b
        scanf("%lld", &b[i]);
    int len_sa=subset(a,len_a,sa,p); // all subsets of a
    int len_sb=subset(b,len_b,sb,p); // all subsets of a
    sort(sb, sb+len_sb);
    // merge same element of sb, assume not empty
    LL num[1<<19], len_sb2=1;
    num[0]=1; //its multiplicity
    for (i=1;i<len_sb;i++) {
        if (sb[i]!=sb[i-1]) { // new element
            sb[len_sb2]=sb[i];
            num[len_sb2]=1;
            len_sb2++;
        }
        else {
            num[len_sb2-1]++;
        }
    }
    LL ans = (sb[0]==1) ? num[0]%p : 0; // the number of 1 in sb2
    // compute 1 in sa and cross the two sides
    // for each x in sa, find its inverse in sb2
    for (i=0; i<len_sa; i++) {
        if (sa[i]==1) ans=(ans+1)%p;
        LL y = exp(sa[i], p-2, p); // inverse
        int it = lower_bound(sb, sb+len_sb2, y) - sb;
        if (it<len_sb2 && sb[it]==y) // found
            ans = (ans + num[it])%p;
    }
    printf("%lld\n", ans);
    return 0;
}

```

這一題當然也可以用 set/map 來做，因為需要合併，我們用 map 來做會比前一支程式方便，時間複雜度則是相同的。這裡的輸入元素放在 vector 中，而子集合窮舉是以遞迴方式寫的，在遞迴到最後時將所產生的子集合乘積放入 map 中。

```
// subset product = 1 mod P,  $O(n \cdot 2^{(n/2)})$ , using map
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
// recursive generate product of subsets of v[0..i]
// current product=prod, result stored in M
void rec(vector<LL> &v, int i, LL prod, map<LL,LL> &M, LL p) {
    if (i>=v.size()) { // terminal condition
        M[prod] += 1; // insert into map
        return;
    }
    rec(v, i+1, (prod*v[i])%p, M, p); // select v[i]
    rec(v, i+1, prod, M, p); // discard v[i]
    return;
}

// find  $x^y \bmod P$ 
LL exp(LL x, LL y, LL p) {
    if (y==0) return 1;
    if (y & 1) return (exp(x, y-1,p)*x)%p;
    // otherwise y is even
    LL t=exp(x, y/2, p);
    return (t*t)%p;
}

int main() {
    int i, n;
    vector<LL> a, b; // input data
    LL p;
    scanf("%d%lld", &n, &p);
    for (i=0;i<n/2;i++) { // half in a
        LL t;
        scanf("%lld", &t);
        a.push_back(t);
    }
    for (i=n/2;i<n;i++) { // half in b
        LL t;
        scanf("%lld", &t);
        b.push_back(t);
    }
    map<LL,LL> M1, M2;
    rec(a,0,1,M1,p); // all subsets of a
    rec(b,0,1,M2,p); // all subsets of b
    M1[1] -= 1; // empty set was counted as product 1
    M2[1] -= 1; // empty set
    LL ans = (M1[1]+M2[1])%p; // the number of 1 in both sides
    // compute 1 cross the two sides
    // for each x in M1, find its inverse in M2
    for (auto e: M1) {
```

```

    LL x=e.first, num=e.second;
    LL y = exp(x, p-2, p); // inverse of x
    //printf("%lld, %lld; ",x,y);
    auto it=M2.find(y);
    if (it!=M2.end()) { // found
        ans = (ans + num*it->second)%p;
    }
}
printf("%lld\n", ans);
return 0;
}

```

最後我們給一個類似的習題，這個題目在上一章也出現過，當時的 $n < 26$ ，現在把他放寬到 38，解法跟前面的例題很類似，注意這裡的 P 不一定也不需要是質數。因為沒有要求模逆元也不必做合併，這題比前面的例題簡單一點，請注意數字的範圍需要使用 long long。

Q-2-10. 子集合的和 (折半枚舉)

輸入 n 個正整數 $A[1..n]$ ，另外給了一個整數 P ，請計算 A 中元素各種組合中，其和最接近 P 但不超過 P 的和是多少。每個元素可以選取或不選取但不可重複選， A 中的數字可能重複。 $A[i]$ 與 P 均不超過 2^{60} ， $0 < n \leq 38$ 。

Time limit: 1 秒

輸入格式：第一行是 n 與 P ，第二行 n 個整數是 $A[i]$ ，同行數字以空白間隔。

輸出格式：最接近 P 但不超過 P 的和。

範例輸入：

```

5 17
5 5 8 3 10

```

範例輸出：

```

16

```

在 P-1-6，我們提出了一個尋找最接近區間和的問題，當時用來展示一些窮舉的技巧，只有提出 $O(n^2)$ 的方法，以下的例題我們來展示如何有效率的解這個問題。

P-2-11. 最接近的區間和 (*)

輸入一個整數序列 ($A[1], A[2], \dots, A[n]$)，另外給了一個非負整數 K ，請計算哪一個連續區段的和最接近 K 而不超過 K 。 n 不超過 10 萬，數字總和不超過 10 億。

Time limit: 1 秒

輸入格式：第一行是 n 與 K ，第二行 n 個整數是 $A[i]$ ，同行數字以空白間隔。

輸出格式：在所有區間和中，最接近 K 但不超過 K 的和。

範例輸入：

```
5 10
5 -5 8 -3 4
```

範例輸出：

```
9
```

題目並沒說輸入的是正數，這裡的輸入數字當然有可能是負的，否則有另外一種解法（下一章會討論）。一個區段可以用兩端點 $[l, r]$ 表示，對於可能的右端 r ，我們要設法找到最好的左端 l ，最好的意思就是讓 $A[l:r]$ 的和最接近 K 而不超過 K ，如此一來，然後對所有的 r 再找出最好的解就是最佳解。所以問題的核心再如何對任一右端計算出最好的解。

在 P-1-6 中我們介紹過前綴和 (Prefix-sum)，令 $ps[i]$ 表示前 i 項的和，為方便起見定義 $ps[0]=0$ 。對任一右端 r ， $sum(A[i:r])=ps[r]-ps[i-1]$ ，因此，要使 $sum(A[i:r])$ 不超過 K 又最接近 K ，就是要找 $ps[i-1] \geq ps[r]-K$ 的最小值。解法呼之欲出了，讓 r 從小往大跑，以資料結構維護好所有 $i < r$ 的 $ps[i]$ ，然後找到 $ps[r]-K$ 的 `lower_bound()`。那麼該用甚麼資料結構呢？因為必須不斷的將 prefix-sum 加入，所以必須以動態的資料結構來處理，`set` 就會符合需要。時間複雜度是 $O(n \log(n))$ ，因為在 `set` 中搜尋是 $O(\log(n))$ 。

我們程式其實很簡單，但必須使用到 `set` (或是其他更複雜的資料結構)，所以這一題基本上算超過 APCS 的考試範圍。

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n, psum=0, k, v;
    scanf("%d%d", &n, &k);
    set<int> S({0}); // record the prefix_sum
    int best=0; // solution of empty range
    for (int r=0; r<n; r++) {
        scanf("%d", &v);
```

```

    psum += v; //prefix-sum(r)
    auto it=S.lower_bound(psum-k);
    if (it!=S.end()) // found
        best=max(best, psum-*it); //currently best
    S.insert(psum); // insert prefix-sum(r)
}
printf("%d\n",best);
return 0;
}

```

以下的習題是二維版本的類似題。給一個提示，對所有列的範圍 $[i, j]$ ，把第 i 列到第 j 列“黏”起來變成一個一維陣列，然後用 P-2-11 的方法去做，這樣的時間複雜度會是 $O(M^2N \log(N))$ 。

Q-2-12. 最接近的子矩陣和 (108 高中全國賽) (*)

輸入一個整數二維矩陣 $A[M][N]$ ，另外給了一個整數 K ，請計算哪一個子矩陣的和，也就是對所有 $1 \leq i \leq j \leq M$ and $1 \leq p \leq q \leq N$ ， $\sum_{s=i}^j \sum_{t=p}^q A[s][t]$ 最接近 K 而不超過 K 。
 $M \leq 50$ 且 $M \times N \leq 300,000$ ，每一個整數的絕對值不超過 3,000。

Time limit: 1 秒

輸入格式：每筆測資的第一行有一個正整數 K ；第二行有兩個正整數 M 與 N 。接下來，由上而下，從左至右，有 M 行輸入，每一行有 N 個整數，每一個整數的絕對值不超過 3,000，代表 $A[s][t]$ ，同行整數間以空格隔開。

輸出格式：在所有子矩陣和中，最接近 K 但不超過 K 的和。

範例輸入：

```

6
2 3
-1 -1 1
2 2 2

```

範例輸出：

```

6

```

以下習題是 108 年高中全國賽某一題的核心要算的，是快速幂的一個應用。

Q-2-13. 無理數的快速幂 (108 高中全國賽, simplified)

若 $s + t\sqrt{2} = (x + y\sqrt{2})^n$ ，其中 x, y, s, t 均為正整數，輸入 x, y 與 n ，請計算並輸出 s 與 t 除以 p 的餘數。 $p=10^9+9$ 且 $x, y, n < p$ 。

Time limit: 1 秒

輸入格式：一行含三個正整數，依序為 x, y 與 n ，以空格隔開。

輸出格式： s 與 t ，中間空一格。

範例輸入：

2 3 2

範例輸出：

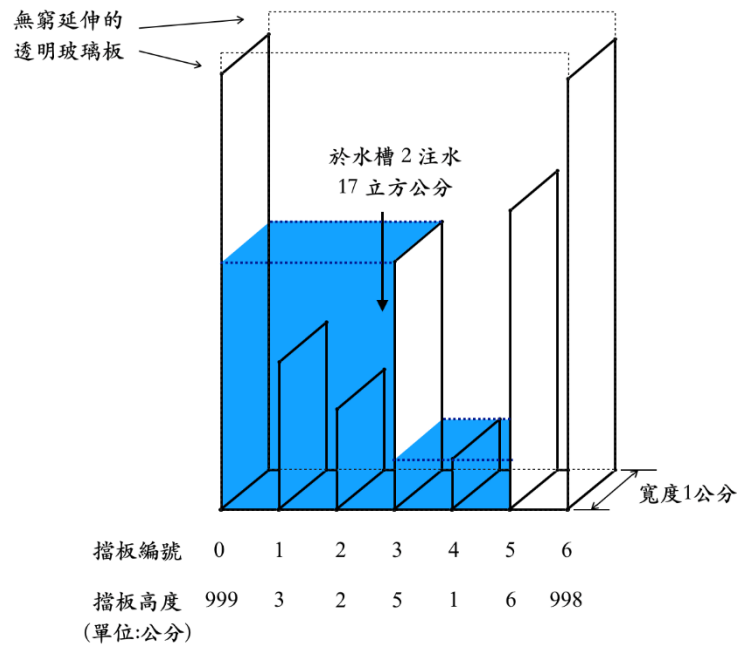
22 12

下一題是 108 高中全國賽的題目，有多種解法，題目需要一些思考，但其實有個漂亮的解法是排序與遞迴的聯合運用，不需要特別的資料結構與方法。這一題有點難，如果做得出來應該已經超過 APCS 五級分的程度了。

Q-2-14. 水槽 (108 高中全國賽) (@@)

我們用一排 n 個擋板建造水槽。擋板的寬度為 1，高度為正整數且均不相同，水槽前後是兩片長寬均為無限大的玻璃板（見下圖例）。相鄰擋板的距離都是 1，故相鄰二擋板之間會形成底面積 1 平方的水槽。

擋板由左而右依序由 0 到 $n - 1$ 編號，第 i 及 $i + 1$ 擋板中間的水槽稱為水槽 i 。現在將總量為 w 立方公分的水緩緩注入水槽 i 。注意水量可能溢出到別的水槽，但是由於所有擋板高度都不同，所以每當溢出時，只會先從一個方向溢出。請計算將總量為 w 立方公分的水緩緩注入水槽 i 後，所有水槽的水深。本題最左的擋板與最右的擋板是所有擋板中最高的兩個，並且保證欲注入的水不會溢出到左右邊界之外；另外，所有水槽的最後水深一定都是整數。以下圖為例，於水槽 2 注入 17 立方公分的水後，各水槽的水深依序為 5, 5, 5, 1, 1, 0。



Time limit: 1 秒

輸入格式：第一行有三個正整數 n ($3 \leq n \leq 10^5$)、 i ($0 \leq i \leq n - 2$) 和 w ($1 \leq w \leq 10^{12}$)，分別代表擋板數、注水水槽編號，及水量。第二行有 n 個以空白間隔的正整數，代表由左到右擋板的高度，每個擋板高度為正整數且不超過 10^9 。請注意注水量可能超過一個 32-bit 整數的範圍。。

輸出格式：輸出為一行，共 $n - 1$ 個整數，依序代表各個水槽水深，數字之間以一個空白間隔。

範例輸入：

```
8 3 27
9 7 5 3 4 6 8 10
```

範例輸出：

```
0 6 6 6 6 3 0
```

Q_2_14 解題提示：我們維護一個水槽高度尚未被決定的區間 $[left, right)$ ，區間以外的水槽高度都已經確定。遞迴函數

$rec(left, right, water, input)$ 計算水量 $water$ 從編號 $input$ 進入後區間的各高度。

如果區間只剩一個水槽，該水槽高度=水量，結束；

否則，在區間中找出最高的隔板，假設此隔板高度為 H 。區間被此隔板分成左右兩邊，考慮下面三種情形：

- $\text{water} \geq (\text{right} - \text{left}) * H$ ，水量足以讓兩邊都至少 H ，區間內所有水槽高度皆是相同的平均值，結束。
- 水量不足以越過輸入這一邊，那麼，另外一邊一定不會有水，縮小區間遞迴呼叫。
- 水量會越過輸入這一邊，那麼，輸入這一邊的水槽高度一定都是 H ，把水量扣掉後遞迴呼叫另外一邊。

為了要找出區間內的最高隔板，我們可以一開始把所有隔板的依照高度從高到低排列，每次要找某區間最高隔板時，我們檢視目前最高的隔板，如果在我們要的區間，那就找到了；如果不在我們要的區間，這個隔板以後也不會用到(想想為何)，可以直接丟掉。因此，只要一開始把隔板排序後依序往後找就可以了，不需要特殊的資料結構，但是要把隔板的高度與位置一起存，所以需要一個兩個欄位資料的排序。

P-2-15. 圓環出口 (APCS202007)

有 n 個房間排列成一個圓環，以順時針方向由 0 到 $n - 1$ 編號。玩家只能順時針方向依序通過這些房間。每當離開第 i 號房間進入下一個房間時，即可獲得 $p(i)$ 點。玩家必須依序取得 m 把鑰匙，鑰匙編號由 0 至 $m-1$ ，兌換編號 i 的鑰匙所需的點數為 $Q(i)$ 。一旦玩家手中的點數達到 $Q(i)$ 就會自動獲得編號 i 的鑰匙，而且手中所有的點數就會被「全數收回」，接著要再從當下所在的房間出發，重新收集點數兌換下一把鑰匙。遊戲開始時，玩家位於 0 號房。請計算玩家拿到最後一把鑰匙時所在的房間編號。

以下是一個例子。有 7 個房間， $p(i)$ 依序是 (2, 1, 5, 4, 3, 5, 3)，其中 0 號房間的點數是 2。假設所需要的鑰匙為 3 把， $Q(i)$ 依序是 (8, 9, 12)。從 0 號房出發，在「離開 2 號房，進入 3 號房」時，獲得恰好 $2+1+5 = 8$ 點，因此在進入 3 號房時玩家兌換到 0 號鑰匙；接著從 3 號房開始繼續累積點數，直到「離開 5 號房，進入 6 號房」時，手中的點數為 12，於是在進入 6 號房時獲得 1 號鑰匙，手中點數再次被清空。最後，從 6 號房出發，直到「離開 3 號房，進入 4 號房」時，方可獲得至少 12 點的點數，來兌換最後一把鑰匙。因此，拿到最後一把鑰匙時所在的房間編號為 4。

Time limit: 1 秒

輸入格式：第一行有兩個正整數 n 與 m ，第二行有 n 個正整數，依序是各房間的點數 $p(i)$ ，第三行有 m 個正整數依序是各鑰匙需要的點數 $Q(i)$ ，。同一行連續二數

字間以空白隔開。n 不超過 $2e5$ ，m 不超過 $2e4$ ， $p(i)$ 總和不超過 $1e9$ ， $Q(i)$ 不超過所有 $p(i)$ 總和。

輸出格式：輸出拿到最後一把鑰匙時所在的房間編號。

範例輸入：

```
7 3
2 1 5 4 3 5 3
8 9 12
```

範例輸出：

```
4
```

簡單說，這個題目就是每次要在一個正整數陣列中，對於某個開始位置，找到一個最小的區間，滿足區間和大於等於某輸入 $Q(i)$ 。關於圓環，有兩個簡單的處理方式，一個是判斷超過結尾時，從頭開始；另外一種方式是將陣列重複兩次，每次找到後除以 n 取餘數。最簡單的搜尋方法就是一個一個往下找，程式也很好寫，但是效率不夠好，以下是範例程式。

```
#include <stdio>
#define N 200010
int p[N];

int main() {
    int i, n, m;
    scanf("%d%d", &n, &m);
    for (i=0; i<n; i++)
        scanf("%d", &p[i]);
    int room=0; // current room
    for (i=0; i<m; i++) {
        int q;
        scanf("%d", &q);
        while (q>0) {
            q -= p[room++];
            if (room==n) room=0; //the next of n-1
        }
        printf("%d\n", room);
    }
    return 0;
}
```

要有效率的搜尋，腦海中浮現的自然是二分搜，但是二分搜必須是在單調序列上才能使用，而且我們要找的是區間和。在 P-1-6 中我們介紹過前綴和 (Prefix-sum)，如果我們把所有房間的點數改成計算出前綴和，那麼正數的前綴和必然是遞增的，而我們要

找的區間和也自然的轉換成找一個前綴和。關於二分搜，我們推薦前面介紹的一路往前跳的寫法，以下是範例程式。第 10 行的迴圈計算前綴和，然後第 13 行的迴圈處理每一次需求的點數，在第 15 行，如果目前房間不是 0，就將所需的點數 q 加上前一個房間的前綴和，這就是我們要搜尋的前綴和；因為本題是圓環，在第 17 行我們判斷是否需求會超過陣列尾端從 0 開始。接著進行二分搜，因為我們要始終保持 $p[\text{room}] < q$ 這個特性，跳躍結束時所在位置是最後一個小於 q 的，所以根據題意，要加 2 才是下一個開始位置。要小心在二分搜開始往前跳之前，需要先檢查目前位置是否已經滿足需求。

```

00 // P-2-15, repeatedly search range sum in circular array
01 #include <stdio>
02 #define N 500010
03 int p[N];
04
05 int main() {
06     int i, n, m;
07     scanf("%d%d", &n, &m);
08     for (i=0; i<n; i++)
09         scanf("%d", &p[i]);
10     for (i=1; i<n; i++)
11         p[i] += p[i-1];
12     int room=0, q, total=p[n-1];
13     for (i=0; i<m; i++) {
14         scanf("%d", &q);
15         if (room != 0) // desired prefix-sum
16             q += p[room-1];
17         if (q > total) // over the end
18             room = 0, q -= total;
19         // find the last one < q
20         if (p[room] >= q) { // current room is enough
21             room = (room+1)%n;
22             continue;
23         }
24         // binary search
25         for (int jump=(n-room)/2; jump>0; jump/=2) {
26             while (room+jump<n && p[room+jump]<q)
27                 room += jump;
28         }
29         room = (room+2)%n; // room+1 is the first >=q
30     }
31     printf("%d\n", room);
32     return 0;
33 }
34

```

我們當然也可以用 STL 的 `lower_bound` 來做二分搜，以下是範例程式，這裡我們以將原來陣列複製一份在後面的方式來處理圓環轉過尾端從開始的問題。其他的寫法都類似。

```

// P-2-15, using extended array and lower_bound
#include <bits/stdc++.h>
using namespace std;
#define N 500010
int p[N];

int main() {
    int i, n, m;
    scanf("%d%d", &n, &m);
    for (i=0; i<n; i++)
        scanf("%d", &p[i]);
    for (i=0; i<n; i++) // double array
        p[n+i] = p[i];
    // prefix sum
    for (i=1; i<2*n; i++)
        p[i] += p[i-1];
    int room=0, q;
    for (i=0; i<m; i++) {
        scanf("%d", &q);
        if (room != 0) // desired prefix-sum
            q += p[room-1];
        // binary search the first >= q
        room = lower_bound(p+room, p+2*n, q) - p;
        room = (room+1)%n; // adjust
    }
    printf("%d\n", room);
    return 0;
}

```